Technology Stack

Date	26 June 2026
Team ID	6a3a555ae5895b2c9759029d
Project Name	EduGenie - Google Gemini Powered Learning Assistant
Maximum Marks	2 Marks

Define Your Technology Stack:

Identify the programming languages, frameworks, databases, front-end tools, back-end tools, and APIs that you will use to build your solution. A well-chosen technology stack ensures that your application is scalable, maintainable, and aligned with your project requirements.

Fill out the table below to detail the specific technologies chosen for each component of your architecture and provide a brief justification for your choice.

Technology Stack Details

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
1	**Frontend / Client-Side** *(e.g., React, Angular, HTML/CSS)*	HTML5, Jinja2 templates, CSS, JS	No build step required; templates render server-side from FastAPI, keeping the stack lightweight for a fast demo turnaround.

2	**Backend / Server-Side** *(e.g., Node.js, Python/Django, Java)*	Python 3.10, FastAPI, Uvicorn	Async-first framework with automatic OpenAPI docs, native Pydantic validation, and fast iteration for REST endpoints.
3	**Database / Data Storage** *(e.g., MySQL, MongoDB, PostgreSQL)*	JSON flat files (data/*.json)	Zero-setup persistence that mirrors a relational schema (Users, Queries, Responses, Quizzes, Summaries, Learning Paths) without requiring an external DBMS for a demo/academic project.
4	**Cloud / Hosting / Deployment** *(e.g., AWS, Azure, Heroku, Docker)*	Docker (python:3.10-slim)	Containerization ensures the app runs consistently across development and evaluation environments; exposed on port 8000.
5	**Version Control & CI/CD**	Git, GitHub	Standard collaborative source control for the team; no automated CI pipeline is configured yet.

	(e.g., GitHub, GitLab, Jenkins)		
6	**Third-Party APIs / Other Tools** *(e.g., Stripe, Twilio, Google Maps)*	Google Gemini API & local LaMini-Flan-T5 model	Combines cloud-grade reasoning quality (Gemini, with dynamic best-model resolution) with an offline-capable local model for resilience.